

CAS PRATIQUE

Programmation Orientee Objet en PHP

Creation d'un Blog avec le CRUD

Matiere : Programmation Web – PHP

Objectif : Appliquer la POO via un projet concret

Notions abordees :

Classe & Constructeur	Encapsulation	Connexion PDO
CREATE	READ	UPDATE & DELETE

Contents

1	Introduction et objectifs	3
1.1	Ce que vous allez construire	3
1.2	Structure des fichiers	3
2	Etape 1 – Preparation de la base de donnees	3
2.1	Creation de la base et de la table	3
2.2	Insertion des donnees de test	4
3	Etape 2 – La classe Database (connexion a la BDD)	5
3.1	Pourquoi une classe pour la connexion ?	5
3.2	Construction pas a pas de la classe Database	6
3.2.1	Etape A : declarer la classe et ses attributs	6
3.2.2	Etape B : ajouter la methode privee de connexion	6
3.2.3	Etape C : ajouter le constructeur	8
3.2.4	Etape D : ajouter la methode publique getConnection()	9
3.3	Recapitulatif – config/database.php final	10
4	Etape 3 – La classe Article (modele metier)	12
4.1	Concept	12
4.2	Code – models/Article.php (structure de base)	12
5	Etape 4 – CREATE : creer un article	14
5.1	Concept	14
5.2	Methode create() – a ajouter dans Article.php	14
5.3	Formulaire de creation – views/form_create.php	15
5.4	Traitement de la creation – create.php	16
6	Etape 5 – READ : lire et afficher les articles	17
6.1	Concept	17
6.2	Methode findAll() – recuperer tous les articles	17
6.3	Methode findById() – recuperer un article par son id	18
6.4	Affichage de la liste – index.php	19
6.5	Vue liste – views/list.php	19
6.6	Vue detail – views/detail.php	21
6.7	Synthese du READ – comment les fichiers se parlent	22
7	Etape 6 – UPDATE : modifier un article	24
7.1	Concept	24
7.2	Methode update() – a ajouter dans Article.php	24
7.3	Formulaire de modification – views/form_edit.php	24
7.4	Script de modification – update.php	25
8	Etape 7 – DELETE : supprimer un article	27
8.1	Concept	27
8.2	Methode delete() – a ajouter dans Article.php	27
8.3	Script de suppression – delete.php	27
9	Recapitulatif – models/Article.php complet	28

10 Synthèse générale	30
10.1 Tableau récapitulatif des notions POO	30
10.2 Tableau récapitulatif des vues	31
11 Exercices	31
11.1 Exercice 1 – Compteur d'articles	31
11.2 Exercice 2 – Pagination	31
11.3 Exercice 3 – Recherche par auteur	31
11.4 Exercice 4 – Confirmation avant suppression	32

1. Introduction et objectifs

Ce cas pratique vous guide pas a pas dans la creation d'un blog minimaliste en PHP en appliquant les concepts de la **Programmation Orientee Objet**. Chaque section introduit une nouvelle notion et l'applique directement dans le code du projet.

1.1. Ce que vous allez construire

Un mini-blog avec les quatre operations essentielles :

- **Create** : publier un nouvel article
- **Read** : afficher la liste et le detail d'un article
- **Update** : modifier un article existant
- **Delete** : supprimer un article

1.2. Structure des fichiers

```
blog/  
|-- config/  
|   |-- database.php      <- Classe de connexion a la BDD  
|  
|-- models/  
|   |-- Article.php       <- Classe Article (CRUD)  
|  
|-- views/  
|   |-- list.php          <- Affichage de la liste  
|   |-- detail.php        <- Affichage d'un article  
|   |-- form_create.php   <- Formulaire de creation  
|   |-- form_edit.php     <- Formulaire de modification  
|  
|-- index.php             <- Page d'accueil (liste)  
|-- create.php             <- Traitement creation  
|-- update.php             <- Traitement modification  
|-- delete.php            <- Traitement suppression
```

Listing 1: Arborescence du projet

2. Etape 1 – Preparation de la base de donnees

Avant d'ecrire du PHP, on prepare la base de donnees. Executez ces instructions SQL dans phpMyAdmin ou dans le terminal MySQL.

2.1. Creation de la base et de la table

```
-- On cree la base si elle n'existe pas encore  
CREATE DATABASE IF NOT EXISTS blog_poo;  
  
-- On dit a MySQL : travaille dans cette base
```

```
USE blog_poo;
```

Listing 2: Etape 1a – Créer la base de données

```
CREATE TABLE IF NOT EXISTS articles (  
    id            INT            AUTO_INCREMENT PRIMARY KEY,  
    titre         VARCHAR(255) NOT NULL,  
    contenu       TEXT          NOT NULL,  
    auteur        VARCHAR(100) NOT NULL,  
    created_at    DATETIME      DEFAULT CURRENT_TIMESTAMP,  
    updated_at    DATETIME      DEFAULT CURRENT_TIMESTAMP  
                        ON UPDATE CURRENT_TIMESTAMP  
);
```

Listing 3: Etape 1b – Créer la table articles

Explication des colonnes :

- `id` : numero unique qui s'incrémente automatiquement a chaque insertion.
- `titre`, `contenu`, `auteur` : les données de l'article.
- `created_at` : date d'insertion, remplie automatiquement par MySQL.
- `updated_at` : date de la dernière modification, mise a jour automatiquement.

2.2. Insertion des données de test

Avant de coder, on insere cinq articles pour avoir des données a afficher et a manipuler des le debut.

```
INSERT INTO articles (titre, contenu, auteur) VALUES  
(  
    'Fally Ipupa enflamme le Stade de France',  
    'Le chanteur congolais Fally Ipupa a realise un concert  
    historique  
    au Stade de France devant plus de 80 000 spectateurs. Une  
    soiree  
    inoubliable marquee par ses plus grands tubes et une scene  
    grandiose. La diaspora africaine etait massivement  
    representee.',  
    'Ezechiel ITEWE'  
),  
(  
    'Le nouvel album de Moise Mbiye est un carton',  
    'Moise Mbiye vient de sortir son dernier album gospel qui  
    cartonne deja dans toute l Afrique centrale. En quelques  
    jours  
    seulement, les titres se retrouvent en tete des ecoutes sur  
    les plateformes de streaming. Un succes largement merite.',  
    'Beloved MADZWA'
```

```
),
(
    'Ferre Gola de retour avec un nouveau projet musical',
    'Après plusieurs mois de silence, Ferre Gola annonce un
      nouveau
      projet musical qui promet de faire parler. Le roi du ndule
      a partage quelques extraits sur ses reseaux sociaux et les
      fans sont deja en effervescence.',
    'Ezechiel ITEWE'
),
(
    'Werrason celebre 30 ans de carriere en grande pompe',
    'Le chef d orchestre Werrason Ngiama a celebre ses 30 ans de
      carriere musicale avec un grand concert a Kinshasa. Des
      artistes venus de toute la sous-region ont participe a
      cette soiree exceptionnelle en hommage au Sorcier du Bois.',
    'Beloved MADZWA'
),
(
    'Koffi Olomide signe un accord avec un label international',
    'Le Quadra Koffi Olomide vient de signer un contrat de
      distribution internationale qui lui permettra de toucher
      un public encore plus large. Une nouvelle etape dans la
      carriere deja legendaire de cet artiste incontournable
      de la musique africaine.',
    'Ezechiel ITEWE'
);
```

Listing 4: Etape 1c – Insérer 5 articles de test

Verification : apres avoir execute ces requetes, lancez `SELECT * FROM articles;` dans MySQL. Vous devriez voir 5 lignes avec les articles ci-dessus.

3. Etape 2 – La classe Database (connexion a la BDD)

3.1. Pourquoi une classe pour la connexion ?

Sans POO, on ecrirait le code de connexion dans chaque fichier PHP qui a besoin de la base. Si les identifiants changent, il faut modifier tous les fichiers. Avec une classe, on écrit la connexion **une seule fois** et on l'utilise partout.

Notions POO de cette section :

- **Classe** : un moule pour créer des objets.
- **Attributs** : les variables qui appartiennent a la classe.
- **Constructeur** : la methode appelee automatiquement quand on écrit `new Database()`.

- **Encapsulation** : on met les données sensibles en `private` pour qu'elles ne soient pas accessibles depuis l'extérieur.

3.2. Construction pas à pas de la classe Database

On construit la classe en quatre étapes pour bien comprendre le rôle de chaque élément.

3.2.1. Etape A : déclarer la classe et ses attributs

```
1 <?php
2
3 class Database
4 {
5     // Ces quatre attributs contiennent les informations
6     // de connexion à la base de données.
7     //
8     // Ils sont déclarés "private" : cela signifie que
9     // seul le code à l'intérieur de cette classe peut
10    // les lire ou les modifier. De l'extérieur, personne
11    // ne peut faire $db->host ou $db->password.
12    // C'est l'encapsulation : on protège les données sensibles.
13
14    private string $host      = "localhost";
15    private string $dbName    = "blog_poo";
16    private string $username  = "root";
17    private string $password  = "";
18
19    // Cet attribut va stocker l'objet PDO une fois connecté.
20    // Le "?" devant PDO signifie qu'il peut valoir null
21    // au départ (avant la connexion).
22    private ?PDO $connection = null;
23 }
24 ?>
```

Listing 5: Etape 2a – Déclarer la classe et ses attributs

Pourquoi private ? Si les attributs étaient public, n'importe quel fichier PHP pourrait écrire `$db->password = "autre_chose"` et casser la connexion. Avec `private`, c'est impossible.

3.2.2. Etape B : ajouter la méthode privée de connexion

```
1 <?php
2
3 class Database
4 {
5     private string $host      = "localhost";
6     private string $dbName    = "blog_poo";
7     private string $username  = "root";
```

```
8     private string $password    = "";
9     private ?PDO    $connection = null;
10
11     // Cette methode effectue la vraie connexion a MySQL.
12     //
13     // Elle est "private" car elle ne sert qu'en interne :
14     // l'utilisateur de la classe n'a pas besoin de l'appeler
15     // directement. Elle est appelee depuis le constructeur.
16     //
17     // On utilise PDO (PHP Data Objects) qui est la methode
18     // moderne et securisee pour parler a une base de donnees.
19     private function connect(): void
20     {
21         try {
22             // Le DSN (Data Source Name) decrit la base :
23             // driver, adresse, nom de la base, encodage
24             $dsn = "mysql:host={$this->host}"
25                 . ";dbname={$this->dbName}"
26                 . ";charset=utf8";
27
28             // On cree l'objet PDO avec les identifiants
29             $this->connection = new PDO(
30                 $dsn,
31                 $this->username,
32                 $this->password
33             );
34
35             // Si une erreur SQL se produit, PDO lancera
36             // une exception au lieu de retourner false
37             // silencieusement. Plus facile a debugger.
38             $this->connection->setAttribute(
39                 PDO::ATTR_ERRMODE,
40                 PDO::ERRMODE_EXCEPTION
41             );
42
43             // Les resultats des SELECT seront retournes
44             // sous forme de tableaux associatifs :
45             // $row['titre'] au lieu de $row[0]
46             $this->connection->setAttribute(
47                 PDO::ATTR_DEFAULT_FETCH_MODE,
48                 PDO::FETCH_ASSOC
49             );
50
51         } catch (PDOException $e) {
52             // En cas d'echec de connexion, on affiche
53             // le message d'erreur et on arrete le script
54             die("Erreur de connexion : " . $e->getMessage());
55         }
56     }
57 }
58 ?>
```


Listing 6: Etape 2b – Methode privee connect()

3.2.3. Etape C : ajouter le constructeur

```
1 <?php
2
3 class Database
4 {
5     private string $host      = "localhost";
6     private string $dbName    = "blog_poo";
7     private string $username  = "root";
8     private string $password  = "";
9     private ?PDO $connection = null;
10
11     // Le constructeur est la methode speciale __construct().
12     // Elle est appelee AUTOMATIQUEMENT des qu'on ecrit :
13     //     $database = new Database();
14     //
15     // On n'a pas a l'appeler manuellement. PHP s'en charge.
16     // Ici, on l'utilise pour lancer la connexion des que
17     // l'objet est cree.
18     public function __construct()
19     {
20         // On appelle notre methode privee de connexion
21         $this->connect();
22     }
23
24     private function connect(): void
25     {
26         try {
27             $dsn = "mysql:host={$this->host}"
28                 . " ;dbname={$this->dbName}"
29                 . " ;charset=utf8";
30
31             $this->connection = new PDO(
32                 $dsn,
33                 $this->username,
34                 $this->password
35             );
36
37             $this->connection->setAttribute(
38                 PDO::ATTR_ERRMODE,
39                 PDO::ERRMODE_EXCEPTION
40             );
41
42             $this->connection->setAttribute(
43                 PDO::ATTR_DEFAULT_FETCH_MODE,
44                 PDO::FETCH_ASSOC
45             );
```

```

46         } catch (PDOException $e) {
47             die("Erreur de connexion : " . $e->getMessage());
48         }
49     }
50 }
51 }
52 ?>

```

Listing 7: Etape 2c – Le constructeur

Ce qui se passe quand on écrit `new Database()` :

1. PHP crée un objet de type `Database`.
2. PHP appelle automatiquement `__construct()`.
3. Le constructeur appelle `connect()`.
4. `connect()` crée la connexion PDO et la stocke dans `$this->connection`.
5. L'objet est prêt à être utilisé.

3.2.4. Etape D : ajouter la méthode publique `getConnection()`

```

1  <?php
2
3  class Database
4  {
5      private string $host      = "localhost";
6      private string $dbName    = "blog_poo";
7      private string $username  = "root";
8      private string $password  = "";
9      private ?PDO $connection = null;
10
11     public function __construct()
12     {
13         $this->connect();
14     }
15
16     private function connect(): void
17     {
18         try {
19             $dsn = "mysql:host={$this->host}"
20                 . " ;dbname={$this->dbName}"
21                 . " ;charset=utf8";
22
23             $this->connection = new PDO(
24                 $dsn,
25                 $this->username,
26                 $this->password
27             );
28

```

```

29         $this->connection->setAttribute(
30             PDO::ATTR_ERRMODE,
31             PDO::ERRMODE_EXCEPTION
32         );
33
34         $this->connection->setAttribute(
35             PDO::ATTR_DEFAULT_FETCH_MODE,
36             PDO::FETCH_ASSOC
37         );
38
39     } catch (PDOException $e) {
40         die("Erreur de connexion : " . $e->getMessage());
41     }
42 }
43
44 // Cette methode est "public" : elle peut etre appelee
45 // depuis n'importe quel fichier PHP.
46 //
47 // Son seul but : retourner l'objet PDO stocke dans
48 // l'attribut prive $connection.
49 //
50 // Ainsi, les autres classes peuvent obtenir la connexion
51 // sans pouvoir la modifier directement.
52 //
53 // Exemple d'utilisation :
54 //     $database = new Database();
55 //     $pdo      = $database->getConnection();
56 public function getConnection(): PDO
57 {
58     return $this->connection;
59 }
60 }
61 ?>

```

Listing 8: Etape 2d – Methode publique getConnection()

Pourquoi ne pas rendre \$connection public directement ? Si \$connection etait public, du code exterieur pourrait ecrire \$database->connection = null et casser la connexion. Avec getConnection(), on retourne la valeur mais on ne laisse pas la modifier. C'est le principe du **getter** en encapsulation.

3.3. Recapitulatif – config/database.php final

Voici le fichier complet, propre et pret a l'emploi.

```

1 <?php
2
3 class Database
4 {
5     private string $host      = "localhost";

```

```
6     private string $dbName      = "blog_poo";
7     private string $username    = "root";
8     private string $password    = "";
9     private ?PDO    $connection = null;
10
11     public function __construct()
12     {
13         $this->connect();
14     }
15
16     private function connect(): void
17     {
18         try {
19             $dsn = "mysql:host={$this->host}"
20                 . " ;dbname={$this->dbName}"
21                 . " ;charset=utf8";
22
23             $this->connection = new PDO(
24                 $dsn,
25                 $this->username,
26                 $this->password
27             );
28
29             $this->connection->setAttribute(
30                 PDO::ATTR_ERRMODE,
31                 PDO::ERRMODE_EXCEPTION
32             );
33
34             $this->connection->setAttribute(
35                 PDO::ATTR_DEFAULT_FETCH_MODE,
36                 PDO::FETCH_ASSOC
37             );
38
39             } catch (PDOException $e) {
40                 die("Erreur de connexion : " . $e->getMessage());
41             }
42         }
43
44     public function getConnection(): PDO
45     {
46         return $this->connection;
47     }
48 }
49 ?>
```

Listing 9: config/database.php – Version finale

4. Etape 3 – La classe Article (modele metier)

4.1. Concept

La classe `Article` represente un article du blog. Elle contient les donnees d'un article (titre, contenu, auteur) et les methodes pour les manipuler en base de donnees.

Injection de dependance : le constructeur de `Article` recoit un objet `PDO` en parametre. On dit que la dependance (la connexion) est *injectee* de l'exterieur. La classe ne cree pas elle-meme la connexion : elle la recoit et l'utilise. Cela rend le code plus propre et plus facile a tester.

4.2. Code – models/Article.php (structure de base)

```
1  <?php
2
3  class Article
4  {
5      // -----
6      // Attributs prives
7      // -----
8      // Chaque attribut represente une colonne de la table.
9      // Ils sont "private" : on y accede uniquement par
10     // les getters et les setters definis ci-dessous.
11
12     private int    $id;
13     private string $titre;
14     private string $contenu;
15     private string $auteur;
16
17     // La connexion PDO, recue depuis l'exterieur
18     private PDO    $db;
19
20     // -----
21     // Constructeur
22     // -----
23     // Parametres :
24     //   $db      : l'objet PDO (connexion a la base)
25     //   $data    : tableau optionnel contenant les valeurs
26     //               de l'article (titre, contenu, auteur)
27     //
28     // Le "= []" signifie que $data est facultatif.
29     // Si on n'en a pas besoin, on peut ecrire juste :
30     //   new Article($pdo)
31     public function __construct(PDO $db, array $data = [])
32     {
33         $this->db = $db;
34
35         // Si des donnees sont fournies, on remplit
36         // les attributs de l'objet
```

```
37         if (!empty($data)) {
38             $this->id      = $data['id']      ?? 0;
39             $this->titre    = $data['titre']   ?? '';
40             $this->contenu  = $data['contenu'] ?? '';
41             $this->auteur   = $data['auteur']  ?? '';
42         }
43     }
44
45     // -----
46     // Getters : lire les attributs prives
47     // -----
48     // Un getter retourne la valeur d'un attribut prive.
49     // Sans getter, ecrire $article->titre declencherait
50     // une erreur car l'attribut est private.
51
52     public function getId(): int
53     {
54         return $this->id;
55     }
56
57     public function getTitre(): string
58     {
59         return $this->titre;
60     }
61
62     public function getContenu(): string
63     {
64         return $this->contenu;
65     }
66
67     public function getAuteur(): string
68     {
69         return $this->auteur;
70     }
71
72     // -----
73     // Setters : modifier les attributs prives
74     // -----
75     // Un setter permet de modifier la valeur d'un
76     // attribut de facon controlee.
77     // On pourrait y ajouter une validation (ex: verifier
78     // que le titre n'est pas vide) avant d'assigner.
79
80     public function setTitre(string $titre): void
81     {
82         $this->titre = $titre;
83     }
84
85     public function setContenu(string $contenu): void
86     {
87         $this->contenu = $contenu;
```

```

88     }
89
90     public function setAuteur(string $auteur): void
91     {
92         $this->auteur = $auteur;
93     }
94
95     // Les methodes CRUD seront ajoutees dans les
96     // etapes suivantes.
97 }
98 ?>

```

Listing 10: models/Article.php – Squelette avec attributs et constructeur

5. Etape 4 – CREATE : creer un article

5.1. Concept

L'operation CREATE consiste a inserer un nouvel article dans la base. On utilise une **requete preparee PDO** : on ecrit d'abord la structure SQL avec des espaces reserves, puis on y injecte les valeurs. Cela protege contre les injections SQL.

5.2. Methode create() – a ajouter dans Article.php

```

1  // Ajouter cette methode dans la classe Article,
2  // apres les setters.
3
4  // -----
5  // CREATE : inserer un nouvel article
6  // -----
7  // Comment ca fonctionne :
8  //   1. On prepare la requete SQL avec des :variables
9  //   2. On lie les vraies valeurs a ces variables
10 //   3. On execute la requete
11 //   4. PDO envoie le SQL a MySQL de facon securisee
12 //
13 // Retourne true si l'insertion a reussi, false sinon.
14 public function create(): bool
15 {
16     // La requete SQL avec des espaces reserves (:titre etc.)
17     // Ces espaces seront remplaces par les vraies valeurs
18     // lors du bindParam, jamais directement dans le SQL.
19     $sql = "INSERT INTO articles (titre, contenu, auteur)
20           VALUES (:titre, :contenu, :auteur)";
21
22     // prepare() compile la requete SQL cote MySQL
23     $stmt = $this->db->prepare($sql);
24
25     // bindParam() lie chaque espace reserve a un attribut
26     // de notre objet. La valeur sera envoyee a l'execution.

```

```

27     $stmt->bindParam(':titre',    $this->titre);
28     $stmt->bindParam(':contenu',  $this->contenu);
29     $stmt->bindParam(':auteur',   $this->auteur);
30
31     // execute() envoie la requete avec les valeurs liees
32     // Retourne true si ok, false si echec
33     return $stmt->execute();
34 }

```

Listing 11: Methode create() – Insertion en base

5.3. Formulaire de creation – views/form_create.php

Ce fichier contient uniquement le HTML du formulaire. On le separe du traitement pour garder le code lisible.

```

1  <!DOCTYPE html>
2  <html lang="fr">
3  <head>
4      <meta charset="UTF-8">
5      <title>Creer un article</title>
6  </head>
7  <body>
8
9  <h1>Rediger un nouvel article</h1>
10
11  <!-- Si une erreur a ete definie dans le script appelant,
12       on l'affiche ici. htmlspecialchars() evite le XSS. -->
13  <?php if (!empty($erreur)): ?>
14      <p style="color: red;"><?= htmlspecialchars($erreur) ?></p>
15  <?php endif; ?>
16
17  <!-- Le formulaire envoie les donnees en POST vers create.php
18       -->
19  <form action="create.php" method="POST">
20
21      <label>Titre de l'article :</label><br>
22      <input type="text" name="titre" required>
23      <br><br>
24
25      <label>Auteur :</label><br>
26      <input type="text" name="auteur" required>
27      <br><br>
28
29      <label>Contenu :</label><br>
30      <textarea name="contenu" rows="8" cols="60" required></
31          textarea>
32      <br><br>
33
34      <button type="submit">Publier l'article</button>

```



```
34 </form>
35
36 <br>
37 <a href="index.php">Retour a la liste</a>
38
39 </body>
40 </html>
```

Listing 12: views/form_create.php – Formulaire HTML

5.4. Traitement de la creation – create.php

```
1 <?php
2
3 // On charge les deux classes dont on a besoin
4 require_once 'config/database.php';
5 require_once 'models/Article.php';
6
7 $erreur = '';
8
9 // On ne traite que si le formulaire a ete soumis en POST
10 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
11
12     // Recuperation des donnees du formulaire
13     // trim() supprime les espaces en debut et fin de chaine
14     $titre = trim($_POST['titre']);
15     $contenu = trim($_POST['contenu']);
16     $auteur = trim($_POST['auteur']);
17
18     // Validation : aucun champ ne doit etre vide
19     if (empty($titre) || empty($contenu) || empty($auteur)) {
20         $erreur = "Tous les champs sont obligatoires.";
21     } else {
22
23         // 1. On cree la connexion
24         $database = new Database();
25         $pdo = $database->getConnection();
26
27         // 2. On cree un objet Article avec les donnees du
28             formulaire
29         $article = new Article($pdo, [
30             'titre' => $titre,
31             'contenu' => $contenu,
32             'auteur' => $auteur,
33         ]);
34
35         // 3. On appelle create() pour inserer en base
36         if ($article->create()) {
37             // Succes : on redirige vers la page d'accueil
38             header('Location: index.php');
39             exit;
```

```

39         } else {
40             $erreur = "Erreur lors de la creation de l'article."
41             ;
42         }
43     }
44
45     // Si on arrive ici (pas de redirection), on affiche le
46     // formulaire
47     // La variable $erreur est disponible dans la vue
48     require_once 'views/form_create.php';
49     ?>

```

Listing 13: create.php – Script de traitement du formulaire

Securite : toujours afficher les donnees venant de l'utilisateur avec `htmlspecialchars()`. Cela convertit les caracteres comme `<` et `>` en entites HTML et empeche l'execution de scripts malveillants (attaque XSS).

6. Etape 5 – READ : lire et afficher les articles

6.1. Concept

L'operation READ sert a lire des donnees dans la base. On construit deux methodes separees car elles repondent a deux besoins differents :

- `findAll()` : recuperer **tous** les articles pour la liste.
- `findById()` : recuperer **un seul** article pour la page de detail.

On les construit une par une.

6.2. Methode `findAll()` – recuperer tous les articles

```

1  // Ajouter cette methode dans la classe Article.
2
3  // -----
4  // READ : recuperer tous les articles
5  // -----
6  // Cette methode retourne un tableau (array) contenant
7  // tous les articles de la base, du plus recent au plus
8  // ancien (ORDER BY created_at DESC).
9  //
10 // Chaque element du tableau est lui-meme un tableau
11 // associatif correspondant a une ligne de la table :
12 // $articles[0]['titre'], $articles[0]['auteur'], etc.
13 //
14 // Si la table est vide, elle retourne un tableau vide [].
15 public function findAll(): array
16 {
17     // On selectionne seulement les colonnes utiles pour

```

```

18 // la liste. Pas besoin du contenu complet ici.
19 $sql = "SELECT id, titre, auteur, created_at
20        FROM articles
21        ORDER BY created_at DESC";
22
23 // prepare() meme sans parametre : bonne pratique
24 $stmt = $this->db->prepare($sql);
25
26 // execute() lance la requete
27 $stmt->execute();
28
29 // fetchAll() recupere TOUTES les lignes d'un coup
30 // et les retourne sous forme de tableau de tableaux
31 return $stmt->fetchAll();
32 }

```

Listing 14: Methode findAll() – SELECT de tous les articles

fetchAll() vs fetch() :

- **fetchAll()** retourne toutes les lignes dans un tableau. On l'utilise quand on veut plusieurs resultats.
- **fetch()** retourne une seule ligne. On l'utilise quand on cherche un enregistrement precis.

6.3. Methode findById() – recuperer un article par son id

```

1 // -----
2 // READ : recuperer un article par son identifiant
3 // -----
4 // Parametre : $id (int) -- l'identifiant de l'article
5 //
6 // Retourne un tableau associatif si l'article existe :
7 //   ['id' => 3, 'titre' => '...', 'contenu' => '...', ...]
8 //
9 // Retourne false si aucun article n'a cet identifiant.
10 // Le type de retour "array/false" exprime cette dualite.
11 public function findById(int $id): array|false
12 {
13     // On cherche l'article dont la colonne id = :id
14     // LIMIT 1 : on s'arrete apres le premier resultat
15     // car l'id est unique, il ne peut y en avoir qu'un
16     $sql = "SELECT *
17            FROM articles
18            WHERE id = :id
19            LIMIT 1";
20
21     $stmt = $this->db->prepare($sql);
22

```

```
23 // PDO::PARAM_INT indique explicitement que :id
24 // est un entier. Cela renforce la securite.
25 $stmt->bindParam(':id', $id, PDO::PARAM_INT);
26
27 $stmt->execute();
28
29 // fetch() retourne une seule ligne ou false
30 return $stmt->fetch();
31 }
```

Listing 15: Methode findById() – SELECT d'un seul article

6.4. Affichage de la liste – index.php

Ce fichier est le point d'entree du blog. Il utilise `findAll()` pour recuperer tous les articles et les passe a la vue.

```
1 <?php
2
3 require_once 'config/database.php';
4 require_once 'models/Article.php';
5
6 // 1. Connexion a la base
7 $database = new Database();
8 $pdo      = $database->getConnection();
9
10 // 2. On cree un objet Article (sans donnees, juste pour
11 //    pouvoir appeler la methode findAll)
12 $articleObj = new Article($pdo);
13
14 // 3. On recupere tous les articles
15 $articles = $articleObj->findAll();
16
17 // 4. On charge la vue qui affiche la liste
18 require_once 'views/list.php';
19 ?>
```

Listing 16: index.php – Point d'entree, recuperation des articles

6.5. Vue liste – views/list.php

La vue recoit la variable `$articles` et l'affiche. Elle ne fait aucun traitement metier : elle affiche uniquement.

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4     <meta charset="UTF-8">
5     <title>Mon Blog</title>
6 </head>
```

```

7 <body>
8
9 <h1>Mon Blog</h1>
10
11 <a href="create.php">Rediger un article</a>
12
13 <hr>
14
15 <?php if (empty($articles)): ?>
16     <!-- Aucun article dans la base -->
17     <p>Aucun article publie pour le moment.</p>
18
19 <?php else: ?>
20
21     <!-- On parcourt le tableau $articles -->
22     <!-- Chaque $article est un tableau associatif -->
23     <?php foreach ($articles as $article): ?>
24
25         <article>
26             <!-- htmlspecialchars() protege contre le XSS -->
27             <h2><?= htmlspecialchars($article['titre']) ?></h2>
28
29             <p>
30                 <small>
31                     Par <strong>
32                         <?= htmlspecialchars($article['auteur']) ?>
33                     </strong>
34                     -- publie le <?= $article['created_at'] ?>
35                 </small>
36             </p>
37
38             <!-- Liens vers les actions possibles -->
39             <p>
40                 <a href="views/detail.php?id=<?= $article['id'] ?>">
41                     Lire la suite
42                 </a>
43                 &nbsp;|&nbsp;
44                 <a href="update.php?id=<?= $article['id'] ?>">
45                     Modifier
46                 </a>
47                 &nbsp;|&nbsp;
48                 <a href="delete.php?id=<?= $article['id'] ?>"
49                     onclick="return confirm('Supprimer cet
50                         article ?');">
51                     Supprimer
52                 </a>
53             </p>
54         </article>

```

```
55         <hr>
56
57         <?php endforeach; ?>
58
59     <?php endif; ?>
60
61 </body>
62 </html>
```

Listing 17: views/list.php – Affichage de la liste des articles

Explication du lien detail : `views/detail.php?id=3` passe l'identifiant de l'article dans l'URL via le parametre `id`. La page detail recuperera cet identifiant avec `$_GET['id']`.

6.6. Vue detail – views/detail.php

Cette page affiche le contenu complet d'un article. Elle utilise `findById()` pour recuperer l'article par son id.

```
1  <?php
2
3  require_once '../config/database.php';
4  require_once '../models/Article.php';
5
6  // Recuperation de l'id depuis l'URL (?id=3)
7  // Le cast (int) force la valeur en entier :
8  // si on recoit "abc", ca devient 0 (invalide)
9  $id = isset($_GET['id']) ? (int) $_GET['id'] : 0;
10
11 // Un id valide est toujours superieur a 0
12 if ($id <= 0) {
13     die("Identifiant d'article invalide.");
14 }
15
16 // Connexion et recherche de l'article
17 $database = new Database();
18 $pdo      = $database->getConnection();
19 $articleObj = new Article($pdo);
20 $article    = $articleObj->findById($id);
21
22 // Si aucun article n'a cet id, findById retourne false
23 if (!$article) {
24     die("Article introuvable.");
25 }
26 ?>
27
28 <!DOCTYPE html>
29 <html lang="fr">
30 <head>
```

```

31 <meta charset="UTF-8">
32 <title><?= htmlspecialchars($article['titre']) ?></title>
33 </head>
34 <body>
35
36 <h1><?= htmlspecialchars($article['titre']) ?></h1>
37
38 <p>
39     <em>
40         Par <?= htmlspecialchars($article['auteur']) ?>
41         -- <?= $article['created_at'] ?>
42     </em>
43 </p>
44
45 <hr>
46
47 <!-- nl2br() convertit les retours a la ligne en <br> -->
48 <p><?= nl2br(htmlspecialchars($article['contenu'])) ?></p>
49
50 <hr>
51
52 <a href="../index.php">Retour a la liste</a>
53 &nbsp;|&nbsp;
54 <a href="../update.php?id=<?= $article['id'] ?>">
55     Modifier cet article
56 </a>
57
58 </body>
59 </html>

```

Listing 18: views/detail.php – Affichage d'un article complet

Explication de nl2br() : la fonction nl2br() convertit les retours a la ligne (\n) en balises
. Sans elle, tout le contenu s'afficherait sur une seule ligne dans le navigateur car HTML ignore les simples retours a la ligne.

6.7. Synthèse du READ – comment les fichiers se parlent

Il est important de comprendre comment les fichiers travaillent ensemble lors d'une operation de lecture. Le schema suivant montre le flux complet.

```
[Navigateur]
|
| Requete HTTP GET vers index.php
v
[index.php]
|
|-- Charge config/database.php --> cree objet Database
|-- Charge models/Article.php   --> cree objet Article
|-- Appelle findAll()
```

```

|           |
|           v
| [MySQL] : SELECT id, titre, auteur, created_at
|           FROM articles ORDER BY created_at DESC
|           |
|           | Retourne un tableau de lignes
|           v
|-- $articles = [ [...], [...], [...], [...], [...] ]
|
|-- Charge views/list.php
|           |
|           | La vue parcourt $articles avec foreach
|           | et affiche chaque article en HTML
|           v
v
[Navigateur]
  Affiche la liste des 5 articles

-----

[Navigateur]
|
| Clic sur "Lire la suite" -> detail.php?id=2
v
[views/detail.php]
|
|-- Lit $_GET['id'] = 2
|-- Appelle findById(2)
|           |
|           v
| [MySQL] : SELECT * FROM articles WHERE id = 2
|           |
|           | Retourne un tableau associatif
|           v
|-- $article = ['id'=>2, 'titre'=>'...', ...]
|
|-- Affiche l'article complet en HTML
v
[Navigateur]
  Affiche le detail de l'article numero 2

```

Listing 19: Flux de lecture – de l'URL a l'ecran

Le role de chaque fichier dans le READ :

- `index.php` : orchestre. Il fait le lien entre la classe et la vue.
- `models/Article.php` : parle a la base de donnees. Il ne sait pas comment afficher.
- `views/list.php` : affiche les donnees. Il ne sait pas comment elles ont ete

obtenues.

- `views/detail.php` : affiche un article et recupere lui-meme son id dans l'URL.

Cette separation des responsabilites rend le code plus facile a maintenir.

7. Etape 6 – UPDATE : modifier un article

7.1. Concept

La modification se deroule en deux temps :

1. **Affichage (GET)** : on recupere l'article et on pre-remplit le formulaire avec ses valeurs actuelles.
2. **Traitement (POST)** : apres soumission, on sauvegarde les nouvelles valeurs en base.

7.2. Methode update() – a ajouter dans Article.php

```

1  // -----
2  // UPDATE : mettre a jour un article existant
3  // -----
4  // On passe l'id en parametre pour cibler le bon article.
5  // Les nouvelles valeurs viennent des attributs de l'objet
6  // (titre, contenu, auteur) mis a jour via les setters.
7  //
8  // Retourne true si la mise a jour a reussi.
9  public function update(int $id): bool
10 {
11     $sql = "UPDATE articles
12           SET     titre     = :titre,
13                 contenu  = :contenu,
14                 auteur   = :auteur
15           WHERE  id       = :id";
16
17     $stmt = $this->db->prepare($sql);
18
19     $stmt->bindParam(':titre',    $this->titre);
20     $stmt->bindParam(':contenu',  $this->contenu);
21     $stmt->bindParam(':auteur',   $this->auteur);
22     $stmt->bindParam(':id',       $id, PDO::PARAM_INT);
23
24     return $stmt->execute();
25 }
```

Listing 20: Methode update() – Mise a jour en base

7.3. Formulaire de modification – views/form_edit.php

```

1  <!DOCTYPE html>
```

```

2 <html lang="fr">
3 <head>
4     <meta charset="UTF-8">
5     <title>Modifier l'article</title>
6 </head>
7 <body>
8
9 <h1>Modifier l'article</h1>
10
11 <!-- Le formulaire envoie en POST vers update.php
12      avec l'id dans l'URL pour savoir quel article modifier -->
13 <form action="update.php?id=<?= $id ?>" method="POST">
14
15     <label>Titre :</label><br>
16     <!-- value="" pre-remplit le champ avec la valeur actuelle
17          -->
18     <input type="text" name="titre"
19           value="<?= htmlspecialchars($article['titre']) ?>"
20           required>
21     <br><br>
22     <label>Auteur :</label><br>
23     <input type="text" name="auteur"
24           value="<?= htmlspecialchars($article['auteur']) ?>"
25           required>
26     <br><br>
27
28     <label>Contenu :</label><br>
29     <textarea name="contenu" rows="8" cols="60" required>
30 <?= htmlspecialchars($article['contenu']) ?>
31     </textarea>
32     <br><br>
33
34     <button type="submit">Enregistrer les modifications</button>
35
36 </form>
37
38 <br>
39 <a href="index.php">Annuler</a>
40
41 </body>
42 </html>

```

Listing 21: views/form_edit.php – Formulaire pre-rempli

7.4. Script de modification – update.php

```

1 <?php
2
3 require_once 'config/database.php';
4 require_once 'models/Article.php';

```

```

5
6 // Recuperation de l'id dans l'URL
7 $id = isset($_GET['id']) ? (int) $_GET['id'] : 0;
8
9 if ($id <= 0) {
10     die("Identifiant invalide.");
11 }
12
13 $database = new Database();
14 $pdo      = $database->getConnection();
15 $articleObj = new Article($pdo);
16
17 // -----
18 // CAS 1 : Le formulaire est soumis (methode POST)
19 // -----
20 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
21
22     $titre    = trim($_POST['titre']);
23     $contenu  = trim($_POST['contenu']);
24     $auteur   = trim($_POST['auteur']);
25
26     if (empty($titre) || empty($contenu) || empty($auteur)) {
27         die("Tous les champs sont obligatoires.");
28     }
29
30     // On utilise les setters pour renseigner
31     // les nouvelles valeurs dans l'objet
32     $articleObj->setTitre($titre);
33     $articleObj->setContenu($contenu);
34     $articleObj->setAuteur($auteur);
35
36     // On execute la mise a jour avec l'id cible
37     if ($articleObj->update($id)) {
38         header('Location: index.php');
39         exit;
40     } else {
41         die("Erreur lors de la mise a jour.");
42     }
43 }
44
45 // -----
46 // CAS 2 : Premiere visite de la page (methode GET)
47 // On affiche le formulaire pre-rempli
48 // -----
49 $article = $articleObj->findById($id);
50
51 if (!$article) {
52     die("Article introuvable.");
53 }
54
55 require_once 'views/form_edit.php';

```

56 ?>

Listing 22: update.php – Affichage et traitement de la modification

8. Etape 7 – DELETE : supprimer un article

8.1. Concept

La suppression est la plus simple des quatre operations. On recoit un identifiant, on verifie qu'il est valide, et on appelle `delete()`.

Attention : une suppression en base est irreversible. En production, on prefere souvent conserver les donnees et juste les masquer (suppression logique). Ici on fait une suppression physique pour rester simple.

8.2. Methode delete() – a ajouter dans Article.php

```
1 // -----
2 // DELETE : supprimer un article par son id
3 // -----
4 // Retourne true si la suppression a reussi.
5 public function delete(int $id): bool
6 {
7     $sql = "DELETE FROM articles WHERE id = :id";
8
9     $stmt = $this->db->prepare($sql);
10    $stmt->bindParam(':id', $id, PDO::PARAM_INT);
11
12    return $stmt->execute();
13 }
```

Listing 23: Methode delete() – Suppression en base

8.3. Script de suppression – delete.php

```
1 <?php
2
3 require_once 'config/database.php';
4 require_once 'models/Article.php';
5
6 // Recuperation et validation de l'identifiant
7 $id = isset($_GET['id']) ? (int) $_GET['id'] : 0;
8
9 if ($id <= 0) {
10     die("Identifiant invalide.");
11 }
12
13 // Connexion a la base
14 $database = new Database();
```

```

15 $pdo          = $database->getConnection();
16 $articleObj   = new Article($pdo);
17
18 // On appelle delete() avec l'id reçu
19 if ($articleObj->delete($id)) {
20     // Suppression réussie : retour à la liste
21     header('Location: index.php');
22     exit;
23 } else {
24     die("Erreur lors de la suppression.");
25 }
26 ?>

```

Listing 24: delete.php – Traitement de la suppression

9. Recapitulatif – models/Article.php complet

```

1  <?php
2
3  class Article
4  {
5      // ---- Attributs privés ----
6      private int    $id;
7      private string $titre;
8      private string $contenu;
9      private string $auteur;
10     private PDO     $db;
11
12     // ---- Constructeur ----
13     public function __construct(PDO $db, array $data = [])
14     {
15         $this->db = $db;
16         if (!empty($data)) {
17             $this->id      = $data['id']      ?? 0;
18             $this->titre   = $data['titre']   ?? '';
19             $this->contenu = $data['contenu'] ?? '';
20             $this->auteur  = $data['auteur']  ?? '';
21         }
22     }
23
24     // ---- Getters ----
25     public function getId(): int    { return $this->id;      }
26     public function getTitre(): string { return $this->titre;
27     }
28     public function getContenu(): string { return $this->contenu
29     ; }
30     public function getAuteur(): string { return $this->auteur;
31     }
32
33     // ---- Setters ----

```

```

31 public function setTitre(string $t): void { $this->titre
    = $t; }
32 public function setContenu(string $c): void { $this->contenu
    = $c; }
33 public function setAuteur(string $a): void { $this->auteur
    = $a; }
34
35 // ---- CREATE ----
36 public function create(): bool
37 {
38     $sql = "INSERT INTO articles (titre, contenu, auteur)
39           VALUES (:titre, :contenu, :auteur)";
40     $stmt = $this->db->prepare($sql);
41     $stmt->bindParam(':titre', $this->titre);
42     $stmt->bindParam(':contenu', $this->contenu);
43     $stmt->bindParam(':auteur', $this->auteur);
44     return $stmt->execute();
45 }
46
47 // ---- READ (tous les articles) ----
48 public function findAll(): array
49 {
50     $sql = "SELECT id, titre, auteur, created_at
51           FROM articles ORDER BY created_at DESC";
52     $stmt = $this->db->prepare($sql);
53     $stmt->execute();
54     return $stmt->fetchAll();
55 }
56
57 // ---- READ (un seul article) ----
58 public function findById(int $id): array|false
59 {
60     $sql = "SELECT * FROM articles WHERE id = :id LIMIT 1";
61     $stmt = $this->db->prepare($sql);
62     $stmt->bindParam(':id', $id, PDO::PARAM_INT);
63     $stmt->execute();
64     return $stmt->fetch();
65 }
66
67 // ---- UPDATE ----
68 public function update(int $id): bool
69 {
70     $sql = "UPDATE articles
71           SET titre = :titre, contenu = :contenu,
72             auteur = :auteur
73           WHERE id = :id";
74     $stmt = $this->db->prepare($sql);
75     $stmt->bindParam(':titre', $this->titre);
76     $stmt->bindParam(':contenu', $this->contenu);
77     $stmt->bindParam(':auteur', $this->auteur);
78     $stmt->bindParam(':id', $id, PDO::PARAM_INT);

```

```

79         return $stmt->execute();
80     }
81
82     // ---- DELETE ----
83     public function delete(int $id): bool
84     {
85         $sql = "DELETE FROM articles WHERE id = :id";
86         $stmt = $this->db->prepare($sql);
87         $stmt->bindParam(':id', $id, PDO::PARAM_INT);
88         return $stmt->execute();
89     }
90 }
91 ?>

```

Listing 25: models/Article.php – Fichier complet avec toutes les methodes

10. Synthese generale

10.1. Tableau recapitulatif des notions POO

Notion	Ou dans le code	Ce qu'elle apporte
Classe	Database, Article	Regroupe des donnees et des comportements.
Constructeur	<code>__construct()</code>	Initialise l'objet automatiquement lors du <code>new</code> .
Encapsulation	Attributs <code>private</code>	Protege les donnees. On y accede par getters/setters.
Getter	<code>getTitre()</code> etc.	Lit un attribut prive depuis l'exterieur.
Setter	<code>setTitre()</code> etc.	Modifie un attribut prive de facon controlee.
Injection	PDO passe au constructeur	La classe recoit la connexion plutot que de la creer.
Requete preparee	<code>prepare()</code> <code>bindParam()</code>	+ Protege contre les injections SQL.
CREATE	<code>create()</code>	Insere un nouvel enregistrement (INSERT).
READ	<code>findAll()</code> , <code>findById()</code>	Lit un ou plusieurs enregistrements (SELECT).
UPDATE	<code>update()</code>	Modifie un enregistrement existant (UPDATE).
DELETE	<code>delete()</code>	Supprime un enregistrement (DELETE).

10.2. Tableau recapitulatif des vues

Les vues sont dans le dossier `views/`. Leur seul role est **d'afficher des donnees**. Elles ne font aucun acces a la base.

Fichier vue	Appelee depuis	Ce qu'elle affiche
<code>views/list.php</code>	<code>index.php</code>	La liste de tous les articles avec les liens Lire, Modifier, Supprimer.
<code>views/detail.php</code>	Elle-meme (auto-portante)	Le contenu complet d'un article. Lit <code>\$_GET['id']</code> directement.
<code>views/form_create.php</code>	<code>create.php</code>	Le formulaire vide pour creer un article.
<code>views/form_edit.php</code>	<code>update.php</code>	Le formulaire pre-rempli pour modifier un article.

Pourquoi separer les vues des scripts ?

Un fichier comme `create.php` fait deux choses : traiter les donnees du formulaire ET afficher le formulaire. Si on melange HTML et PHP dans le meme fichier, le code devient vite illisible.

En separant :

- `create.php` s'occupe uniquement de la logique (valider, inserer, rediriger).
- `views/form_create.php` s'occupe uniquement de l'affichage HTML.

Chaque fichier a une seule responsabilite. C'est plus simple a lire, a corriger et a faire evoluer.

11. Exercices

11.1. Exercice 1 – Compteur d'articles

Ajoutez une methode `countAll(): int` dans la classe `Article` qui retourne le nombre total d'articles en base en utilisant `SELECT COUNT(*) FROM articles`. Affichez ce nombre en haut de la page `index.php`.

11.2. Exercice 2 – Pagination

Modifiez `findAll()` pour accepter deux parametres : `$limite` (nombre d'articles par page) et `$offset` (a partir de quel article on commence). Utilisez `LIMIT` et `OFFSET` dans la requete SQL pour afficher 2 articles par page.

11.3. Exercice 3 – Recherche par auteur

Ajoutez une methode `findByAuteur(string $auteur): array` qui retourne tous les articles d'un auteur donne. Ajoutez un lien sur le nom de l'auteur dans `list.php` pour filtrer les articles par cet auteur.

11.4. Exercice 4 – Confirmation avant suppression

Plutôt que de supprimer directement depuis un lien, créez une page `views/confirm_delete.php` qui affiche le titre de l'article et deux boutons : "Confirmer la suppression" et "Annuler". La suppression n'est effectuée qu'après confirmation via un formulaire POST.